```
/* ############################  SPI Master Code ############################ */


#include <xc.h>

#include <stdint.h>


// CONFIG

#pragma config FOSC = HS        // Oscillator Selection bits (HS oscillator)

#pragma config WDTE = OFF       // Watchdog Timer Enable bit (WDT disabled)

#pragma config PWRTE = OFF      // Power-up Timer Enable bit (PWRT disabled)

#pragma config BOREN = ON       // Brown-out Reset Enable bit (BOR enabled)

#pragma config LVP = OFF        // Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit (RB3 is digital I/O, HV on MCLR must be used for programming)

#pragma config CPD = OFF        // Data EEPROM Memory Code Protection bit (Data EEPROM code protection off)

#pragma config WRT = OFF        // Flash Program Memory Write Enable bits (Write protection off; all program memory may be written to by EECON control)

#pragma config CP = OFF         // Flash Program Memory Code Protection bit (Code protection off)



#define _XTAL_FREQ 4000000

//---------------------------------

// IO Pins Defines (Mappings)

#define UP RB0

#define Down RB1

#define Send RB2

//---------------------------------

// Functions Declarations

void SPI_Master_Init();
```

```
void SPI_Write(uint8_t);
//--------------------------------
// Main Routine
void main(void)
{
 //--[ Peripherals & IO Configurations ]--
 SPI_Master_Init(); // Initialize The SPI in Master Mode @ Fosc/64 SCK
 uint8_t Data = 0; // The Data Byte
 TRISB = 0x07; // RB0, RB1 & RB2 Are Input Pins (Push Buttons)
 TRISD = 0x00; // Output Port (4-Pins)
 PORTD = 0x00; // Initially OFF
 //---------------------------
 while(1)
 {
  if (UP) // Increment The Data Value
  {
   Data++;
   __delay_ms(250);
  }
  if (Down) // Decrement The Data Value
  {
   Data--;
   __delay_ms(250);
  }
  if (Send) // Send The Current Data Value Via SPI
  {
   SPI_Write(Data);
   __delay_ms(250);
```

```
   }
   PORTD = Data; // Display The Current Data Value @ PORTD
  }
  return;
}
//------------------------------
// Functions Definitions

void SPI_Master_Init()
{
  // Set Spi Mode To Master + Set SCK Rate To Fosc/64
  SSPM0 = 0;
  SSPM1 = 0;
  SSPM2 = 0;
  SSPM3 = 0;
  // Enable The Synchronous Serial Port
  SSPEN = 1;
  // Configure The Clock Polarity & Phase (SPI Mode Num. 1)
  CKP = 0;
  CKE = 0;
  // Configure The Sampling Time (Let's make it at middle)
  SMP = 0;
  // Configure The IO Pins For SPI Master Mode
  TRISC5 = 0; // SDO -> Output
  TRISC4 = 1; // SDI -> Input
  TRISC3 = 0; // SCK -> Output
  // If Interrupts Are Needed, Un-comment The Line Below
  // SSPIE = 1; PEIE = 1; GIE = 1;
```

```
}


void SPI_Write(uint8_t Data)
{
  SSPBUF = Data; // Transfer The Data To The Buffer Register
}
```